Conversational Recommender System

A multi-turn LLM shop assistant for phones & headphones — re-architected from a structured pipeline into a hybrid tool-calling agent, proven on an evaluation gate

Group 26

Applied Generative AI · SS 2026

Arthur Galois · Jakob Meltzer · Sofie Parkesaas · Muhammad Azeem Shahzad

Abstract

We build a conversational recommender that acts as a shop assistant for tech products, and take it from prototype to a deployable design. v1 is a structured LangGraph pipeline: split NLU (router + slot-filling) guarded by schema validation and entity resolution, vague language (“cheap”, “good camera”) grounded in dataset percentiles, hybrid pandas + dense-embedding retrieval, and TOPSIS ranking. Adding intents one-by-one proved brittle (whack-a-mole), so we built an evaluation harness — persona conversation simulation + LLM-judge + adversarial/safety suites — and re-architected the dialogue brain into a hybrid tool-calling AGENT over the same deterministic core. On the shared gate the agent roughly DOUBLES conversation quality (0.41->0.81), reaches 100% on adversarial/safety, fixes count/compound requests, and runs faster — with no per-scenario code. The pipeline is retained as a fallback.

Initial Situation & Problem

Traditional recommenders work silently (ranked lists, “people also bought”). Users often start vague and refine as they talk. A conversational recommender (CRS) must instead elicit preferences in real time and adapt.

Core challenges / questions:

- Robust intent detection & preference elicitation from messy natural language.
- Reasoning over a dialogue state to pick the next best action — not blindly answering.
- Safely translating conversational critiques (“cheaper but better camera”) into DB queries.
- What information does the system actually need to reliably recommend a set of items?

Sample Use Cases

1 · Specific lookup

“A Samsung Android phone with 8GB RAM”

Pre-fills brand+OS+spec, recommends immediately — ranked, no needless questions.

2 · Guided exploration

“I want a smartphone”

Asks a focused sequence: OS -> price -> battery -> camera -> RAM, with ‘any/skip’.

3 · Refinement / critique

“cheaper ones” · “bigger battery”

Anchors relative terms to the last results; refines are additive; ‘forget X’ removes.

4 · Vague & vibe language

“cheap phone for gaming”

Maps ‘cheap’->p25 price; ‘gaming’ via semantic embeddings over spec descriptions.

Approach — From Pipeline to Tool-Calling Agent

v2: a hybrid tool-calling agent over a deterministic core (the v1 pipeline below, retained as fallback)

MEMORY
per-user profiles · ‘use my usual’ · personalised greeting

RANKING
TOPSIS (MCDM) <-> learned LTR · semantic blend · feedback flywheel

OBSERVABILITY
LangSmith tracing · per-turn JSONL · intent & funnel analytics

IMAGES
offline enrichment -> GSMArena/Wikipedia -> image_url -> cards

Key Design Decisions

- Split NLU into a small router + a slot extractor instead of one fragile mega-prompt.
- Validate every LLM-proposed slot against a schema; drop unknown / out-of-range / wrong-type.
- Entity resolution as a real layer (alias dictionary + fuzzy match), not ever-growing prompt rules.
- Explicit DialogueState (TypedDict) + LangGraph; information-gathering gate decides ask vs. recommend.
- Confidence-gated clarification; out-of-scope intent answers honestly instead of misfiring.
- Hybrid retrieval: exact pandas filters + dense embeddings (model2vec) for ‘vibe’ queries.
- Transparent TOPSIS ranking; pluggable learned ranker trained from click feedback.

Models & Tools

Engine	Tool-calling agent (default) <-> LangGraph pipeline (fallback)
Tools	search · details · compare · explain · top-picks · catalog
LLM	OpenRouter · GPT-4o-mini (function-calling) · local Ollama opt.
Embeddings	model2vec · potion-base-8M (static, no GPU)
Ranking	TOPSIS (Hwang & Yoon) · numpy logistic LTR
Data / UI	pandas · 2×500 catalog · Streamlit chat + cards + radar
Observability	LangSmith · per-turn JSONL · tokens & latency
Evaluation	persona sim + LLM-judge + 28 adversarial/safety checks

Conversational Capabilities — intents -> system actions

explore
category only -> guided questions (OS->price->battery->camera->RAM)

specific
concrete criteria -> recommend or one narrowing question

refine
react to results: ‘cheaper’ anchored to last set; additive merges

summarize
recap understood preferences in plain language

done
wrap-up / pick one / restart -> clean close + new-search

chitchat
greetings/thanks -> warm reply, product state preserved

ambiguous
too vague -> ask to clarify (confidence-gated)

out_of_scope
photos/buy/stock/warranty -> honest ‘can’t, but I can…’

Cross-cutting: skip (‘any’) · undo (‘ignore that’) · multi-intent · category-switch · vague-term grounding (percentiles) · persistent memory · quick-reply buttons.

Results, Demo & Learnings

0.806	100%	4.6s	95%	6/6	52%
Conversation quality (was 0.412)	Adversarial & safety (was 82%)	Latency / turn (was 7.21s)	Intent routing accuracy	count+compound (was 1/6)	Input tokens cached

Same engine-agnostic gate, both engines: 28 adversarial/safety checks + 10 persona conversations (LLM-judge) + deterministic suites + latency/cost. python eval_harness.py

Demo — Example Conversation

You
suggest me 2 best phones

Assistant
Two top picks by value: Xiaomi Redmi Note 12 Pro+ and the Explorer. (count = 2 honoured)

You
compare them

Assistant
Side-by-side: the Pro+ leads on RAM & battery; the Explorer is cheaper. Cards below.

You
compare it with the iPhone 15 Pro Max

Assistant
I don't carry the iPhone 15 Pro Max — the closest I have is the iPhone 15 Pro. (no fabrication)

Learnings & Limitations

What worked

- Build the eval gate FIRST: persona sim + LLM-judge turned “feels better” into a number.
- The tool-calling agent fixed open-world gaps (counts, compound, anaphora) with NO per-scenario code.
- Grounding lives in deterministic tools; the LLM only plans -> honest, no fabricated specs.
- Hybrid wins: deterministic guardrails (undo, brand-exclude) + an LLM brain beat either alone.

What was hard / we'd change

- Enumerating intents was whack-a-mole — fixing one scenario broke another; the agent removed the ceiling.
- Optimising latency over-tightened replies and dropped quality — the gate caught it and we re-balanced.
- Agent is cloud-first: function-calling on small local models is unreliable (a known trade-off).
- A few personas are capped by the dataset (obscure products), not the system — richer data would lift them.

Evaluation Methodology

- Engine-AGNOSTIC gate: judges observable behaviour, not internal labels -> fairly A/B-tests any engine.
- 28 adversarial/safety checks (count, compound, correction, scope, prompt-injection, PII, i18n, …) plus deterministic resolution / validation / intent suites.
- 10 persona conversations: an LLM ‘customer’ drives multi-turn dialogues; an LLM-judge scores goal / honored / grounded / helpful / safe. Plus per-turn latency + token cost.
- Reproducible: `python eval_harness.py` (agent) vs `ENGINE=pipeline …` (baseline). The gate drove the whole re-architecture and caught real regressions before they shipped.

Reflection — What does the system need?

- The product category — gates the schema, question order, and ranking weights.
- Structured preferences — extracted directly (specific) or elicited via questions (explore), then validated against the schema.
- An anchor for relative critiques — the previous recommendation’s distribution, to ground ‘cheaper’ / ‘bigger battery’.
- A multi-criteria scoring function — TOPSIS over normalised specs, upgradable to a learned ranker.
- Dataset statistics — percentiles that turn vague language (‘cheap’, ‘premium’) into concrete bounds.